SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Dharshavardhan SB	**Reg. No.:**	192420039
Date:		**Duration:**	60 minutes

Code of Conduct

*I **Dharshavardhan SB 192420039** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Dharshavardhan

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with	Inappropriate architecture selected or

	justification aligned to system requirements.		limited justification.	justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

SMART CONFERENCE PAPER SUBMISSION AND REVIEW SYSTEM

ARCHITECTURE DESIGN REPORT

1. Introduction

Academic conferences receive research papers from authors across different institutions and research areas. Conference organizers must coordinate submissions, reviewer assignments, conflict-of-interest checks, reviews, revisions, decisions, and final camera-ready papers. Managing these activities manually can cause delays, duplicated work, missed deadlines, and errors in communication.

The Smart Conference Paper Submission and Review System is a centralized web-based platform designed to manage the complete paper-review lifecycle. It provides role-based access for authors, reviewers, program chairs, and administrators. The system maintains submission records, protects confidential information, tracks deadlines, and generates conference reports.

2. Problem Statement

Manual conference workflows become difficult to manage when the number of papers, reviewers, and conference tracks increases. Paper versions may be misplaced, reviewers may be assigned papers that create conflicts of interest, and decisions or deadlines may not be communicated consistently. The proposed system automates these activities and stores conference information in a centralized and auditable environment.

3. Aim and Objectives

Aim: To design a secure and reliable system for managing research-paper submission, peer review, revision, decision notification, and final paper collection.

- Manage author, reviewer, chair, and administrator registrations.
- Accept paper details and manuscript files with submission validation.
- Allocate papers to suitable reviewers based on expertise, workload, and availability.
- Record and enforce conflict-of-interest declarations before assignment.
- Collect structured review reports and track review completion.
- Manage revision uploads and preserve earlier submission versions.
- Support acceptance, rejection, and revision decisions by authorized chairs.
- Collect final camera-ready papers from accepted authors.
- Send automated notifications and deadline reminders.
- Generate conference analytics, status summaries, and review reports.

4. System Requirements

4.1 Functional Requirements

- User management: registration, login, password reset, profile editing, and role-based permissions.
- Paper submission: enter title, abstract, keywords, authors, track, and upload manuscript files.
- Reviewer management: maintain reviewer expertise, availability, workload, and assignment status.
- Conflict management: capture institutional, personal, financial, and collaboration conflicts; block conflicted assignments.
- Review workflow: submit scores, recommendations, comments for authors, and confidential comments for chairs.
- Revision tracking: request revisions, set deadlines, upload revised versions, and retain version history.
- Decision management: chairs review recommendations and record the final decision.
- Camera-ready collection: accept final files and verify that required metadata and documents are present.
- Notification service: send submission receipts, review invitations, reminders, decisions, and deadline alerts.
- Reporting: show submission counts, review completion, acceptance rates, track-wise statistics, and overdue tasks.

4.2 Non-Functional Requirements

- Security: protect accounts, manuscript files, reviews, and decisions through authentication, authorization, encryption, and audit logs.
- Performance: provide timely responses for common actions and support concurrent users during submission deadlines.
- Scalability: allow application instances and background workers to scale as paper volume increases.
- Availability: minimize downtime and recover services after failures.
- Reliability: prevent data loss and maintain consistent paper, review, and decision records.
- Maintainability: separate presentation, business rules, and data access to simplify testing and upgrades.
- Usability: provide clear dashboards, status indicators, and accessible forms for each user role.
- Confidentiality: reveal reviewer identities and confidential comments only to authorized roles, according to conference policy.

5. Proposed Software Architecture

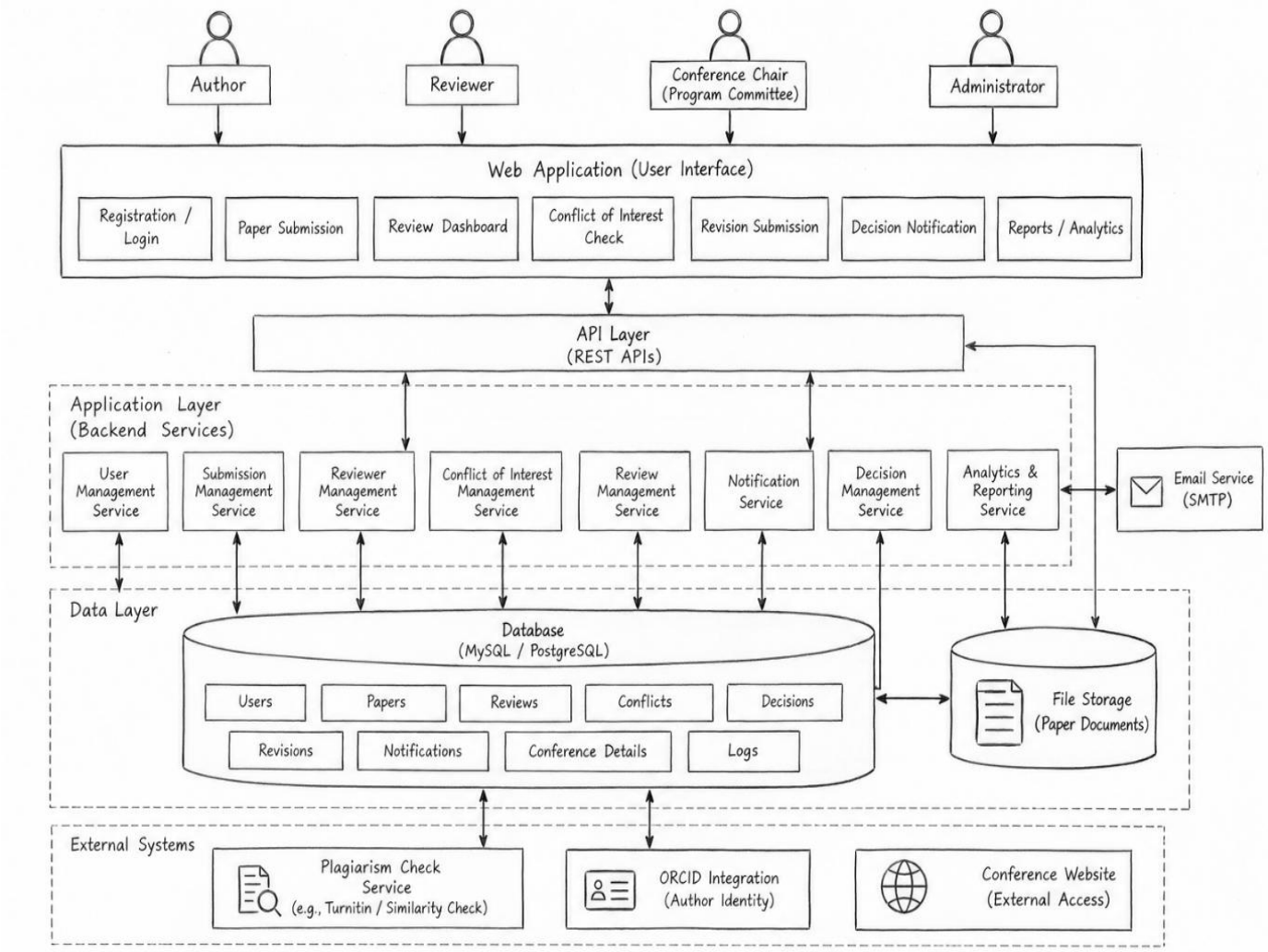
5.1 Selected Architecture: Layered Client–Server Architecture

A layered client–server architecture is proposed. Users access the system through a browser-based interface. Requests are handled by an application layer that contains authentication, submission, reviewer

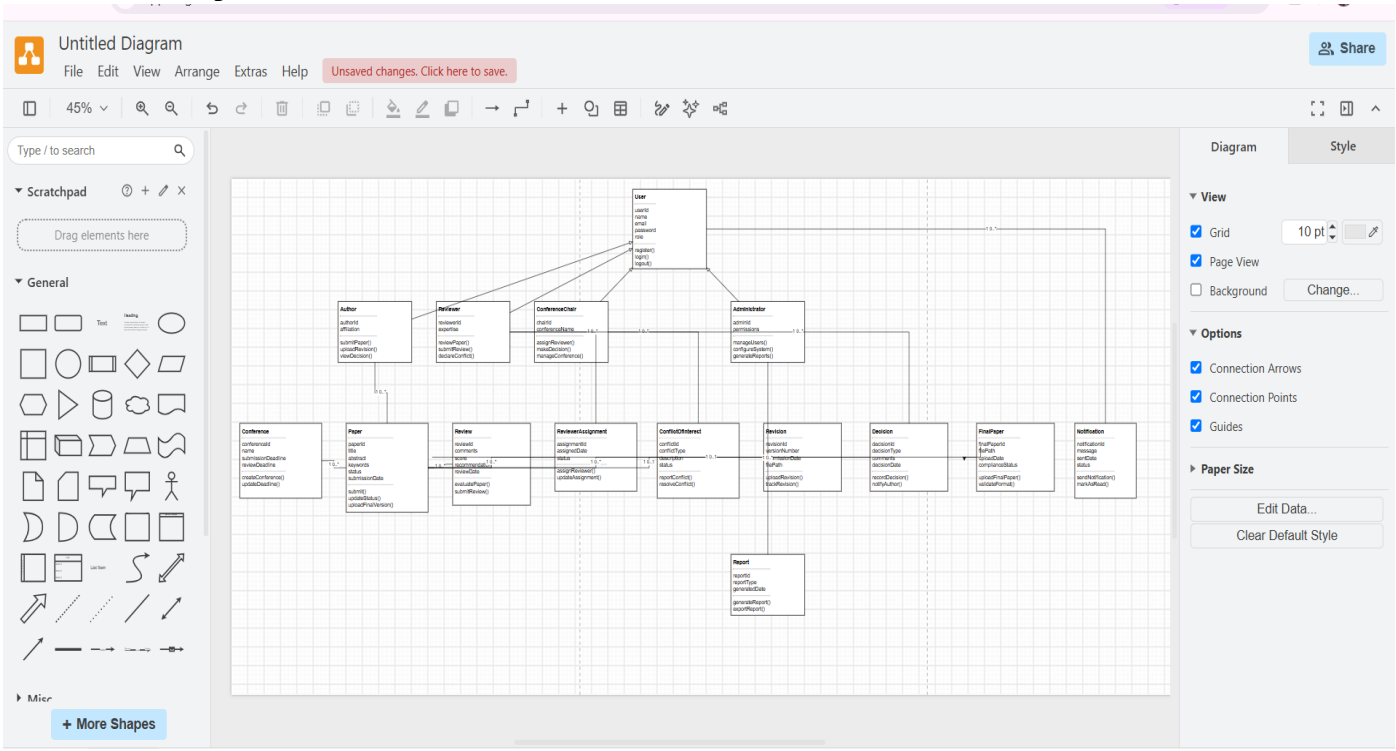
allocation, conflict checks, review, decision, notification, and reporting services. A data-access layer communicates with a relational database and secure file storage. A background job service handles email notifications and scheduled deadline reminders.

This architecture is suitable because the system has clear user roles and workflow stages, requires centralized records, and needs strong access control. It is simpler to implement and maintain than a fully distributed microservices architecture for an academic project, while still allowing individual services to be separated later if demand grows.

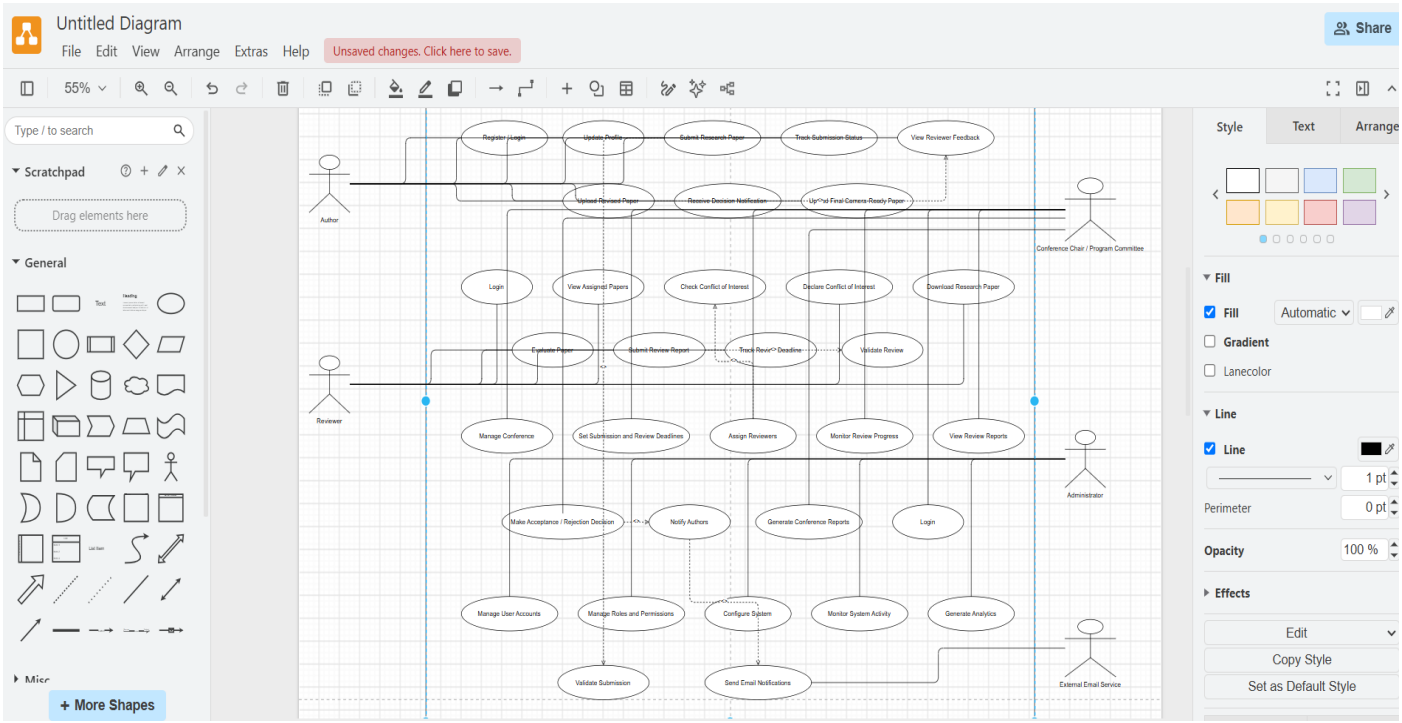
5.2 Architecture Diagram



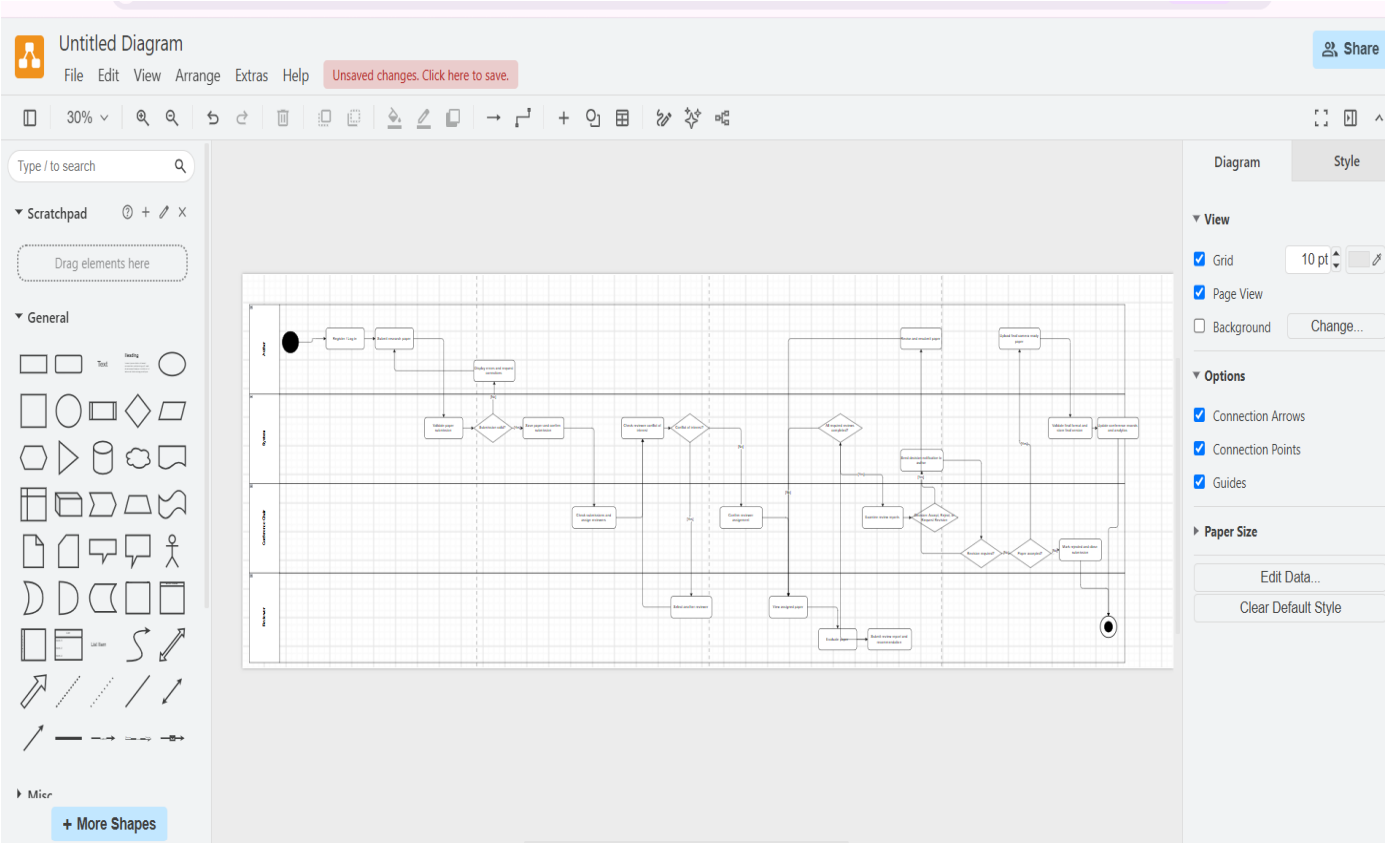
UML Class diagram:



USE CASE DIAGRAM:



ACTIVITY DIAGRAM:



6. Components and Responsibilities

6.1 User Interface

Provides separate dashboards for authors, reviewers, chairs, and administrators. It displays submission status, pending reviews, deadlines, and available actions.

6.2 Authentication and Authorization

Verifies user identity and checks role permissions for every protected operation. It supports secure sessions and prevents unauthorized access to papers and reviews.

6.3 Submission Management

Validates submission metadata, accepts manuscript uploads, assigns a unique paper ID, records timestamps, and preserves version history.

6.4 Reviewer Allocation

Matches papers to reviewers using expertise, declared availability, workload limits, and chair oversight. The system checks conflicts before confirming an assignment.

6.5 Conflict-of-Interest Management

Stores declared conflicts and relationship information. It blocks assignments where a conflict is known and allows users to report a newly discovered conflict. Chairs can reassign the paper.

6.6 Review Management

Provides review forms, scoring criteria, recommendation fields, review deadlines, and status tracking. Reviewer identity and confidential comments are restricted according to policy.

6.7 Revision and Decision Management

Allows authorized chairs to request revision or record acceptance/rejection. Authors can upload revisions before the deadline, and previous versions remain available to authorized users.

6.8 Notification Service

Sends confirmations, invitations, reminders, revision requests, and decision notices. A queue and retry mechanism help handle temporary email-provider failures.

6.9 Reporting and Analytics

Summarizes submissions by track, reviewer workload, pending reviews, decision distribution, acceptance rate, and overdue activities. Access to reports is role-controlled.

6.10 Database and File Storage

The relational database stores users, papers, assignments, reviews, decisions, deadlines, and audit events. Manuscript files are stored separately in protected storage with access checks.

7. Workflow of the System

- The author registers or signs in, completes the paper metadata, uploads the manuscript, and submits it.
- The system validates required fields, creates a paper ID, records the submission time, and sends a receipt.
- The program chair checks paper scope and configures reviewers for the relevant track.
- The allocation service evaluates expertise and workload, checks declared conflicts, and proposes eligible reviewers.
- Reviewers accept or decline invitations and submit their reviews before the deadline.
- The chair examines the submitted reviews and records a decision such as accept, reject, or revise.
- The system notifies the author and, when required, opens a revision window.
- The author submits a revised manuscript; the system stores the new version and may route it for another review.
- Accepted authors upload the camera-ready version. The chair or administrator checks required files and closes the submission.
- The reporting module generates final statistics and preserves authorized records for conference administration.

8. Conflict of Interest and Confidentiality

Conflict checks must happen before an assignment is confirmed and should be repeated if a reviewer reports a new conflict. Examples include recent co-authorship, shared institutional affiliation, supervisor–student relationships, close personal relationships, and financial interests. A conflicted reviewer must not review or influence the decision for that paper. The chair should reassign the paper to an eligible reviewer. Confidentiality is enforced through role-based permissions, secure file access, encrypted network communication, and audit trails. Reviewers can access only assigned papers; authors can access their own submissions and released feedback; chairs can access records needed for their duties; administrators manage configuration without automatically receiving unrestricted review access. The exact visibility rules should follow the conference's published review model.

9. Quality Attribute Analysis

Scalability

The web application can run multiple instances behind a load balancer. Notification and report-generation tasks can run in background workers so that submission requests remain responsive.

Security

Use secure password hashing, multi-factor authentication where available, least-privilege roles, HTTPS, input validation, protected file links, and audit logging.

Performance

Index frequently searched database fields, paginate large lists, validate uploads efficiently, and move email delivery and heavy reports to background jobs.

Reliability

Use database transactions for critical state changes, scheduled backups, file integrity checks, retries for transient notification failures, and tested recovery procedures.

Maintainability

Keep modules and business rules separated, document APIs, use automated tests, and maintain configuration separately from source code.

Availability

Deploy with health checks, monitoring, backups, and a recovery plan. Clearly communicate planned maintenance and provide safe retry options.

Usability

Use role-specific dashboards, clear status labels, form validation, accessible design, and reminders that show the relevant deadline.

Auditability

Record important events such as assignment, review submission, decision changes, and file-version creation with timestamps and actor IDs.

10. Database Design (Main Entities)

- User(UserID, Name, Email, PasswordHash, Role, Affiliation)
- Conference(ConferenceID, Name, SubmissionDeadline, ReviewDeadline, Status)
- Paper(PaperID, ConferenceID, Title, Abstract, Track, Status, CorrespondingAuthorID)
- PaperAuthor(PaperID, UserID, AuthorOrder)
- PaperVersion(VersionID, PaperID, FilePath, VersionNumber, UploadedAt)
- ReviewerProfile(UserID, Expertise, Availability, WorkloadLimit)
- Conflict(ConflictID, ReviewerID, AuthorID or Institution, ConflictType, Status)
- ReviewAssignment(AssignmentID, PaperID, ReviewerID, Status, AssignedAt, DueDate)
- Review(ReviewID, AssignmentID, Score, Recommendation, AuthorComments, ConfidentialComments, SubmittedAt)
- Decision(DecisionID, PaperID, ChairID, DecisionType, Comments, DecisionAt)
- Notification(NotificationID, UserID, Type, Status, SentAt)
- AuditLog(LogID, ActorID, Action, ObjectType, ObjectID, Timestamp)

Primary and foreign keys should enforce relationships. Unique constraints should prevent duplicate assignments where prohibited. Database permissions and application checks must ensure that confidential comments and reviewer identities are not exposed to unauthorized users.

11. Technology Stack (Suggested)

- Frontend: HTML, CSS, JavaScript, and a responsive UI framework.
- Backend: Python Django or Flask, Java Spring Boot, or Node.js; choose one stack for the implementation.
- Database: PostgreSQL or MySQL for structured conference records.
- File storage: protected server storage or private object storage with expiring access links.
- Notifications: SMTP or a trusted email delivery service.
- Deployment and operations: HTTPS, environment-based configuration, automated backups, application logs, and monitoring.

These are proposed implementation choices rather than mandatory technologies. The final stack should reflect the development team's skills, hosting environment, budget, and institutional requirements.

12. Architectural Decisions and Trade-offs

A layered architecture centralizes business rules and is easier to test than a collection of independently deployed microservices. Its main limitation is that a single application can become harder to scale if all modules grow substantially. The design addresses this by keeping module boundaries clear and moving notifications and long-running reports to background workers.

A relational database is suitable because submissions, authors, assignments, reviews, and decisions have structured relationships and require consistency. Separate file storage avoids placing large manuscript binaries directly in database records. Automated reviewer matching can reduce administrative work, but final assignments and decisions should remain subject to authorized human oversight.

Confidentiality controls may make some administrative tasks less convenient, but they are necessary to preserve reviewer independence and trust. Analytics should use aggregated information wherever individual-level details are not required.

13. Testing and Validation

- Functional testing: verify registration, submission, assignment, review, revision, decisions, and final upload.
- Authorization testing: confirm that authors cannot view other authors' papers and reviewers cannot view unassigned papers.
- Conflict testing: verify that known conflicted reviewers cannot be assigned and that newly reported conflicts trigger reassignment.
- Workflow testing: verify deadline states, review completion, revision rounds, and decision notifications.
- Performance testing: simulate concurrent submissions near the deadline and multiple reviewers using the system.
- Recovery testing: restore database backups and verify that uploaded files remain linked to the correct papers.
- Usability testing: ask representative authors, reviewers, and chairs to complete common tasks and report confusing steps.

14. Expected Outcomes and Future Enhancements

The system is expected to streamline conference administration, reduce repetitive manual work, improve reviewer coordination, reduce submission errors, and maintain centralized records. Deadline reminders and status dashboards can make the workflow more transparent to users without exposing confidential reviewer information.

Future enhancements may include expertise-based reviewer recommendations, plagiarism-checking integrations, ORCID sign-in, conference-track support, multilingual interfaces, accessibility improvements, and export of proceedings metadata. Any external integration should follow privacy, consent, security, and conference-policy requirements.

15. Conclusion

The Smart Conference Paper Submission and Review System provides a structured solution for the end-to-end conference paper lifecycle. The proposed layered client–server architecture separates the interface, business workflow, data access, database, file storage, and notification processes. Role-based permissions, conflict checks, version tracking, audit logs, and deadline monitoring address the main requirements for a dependable conference workflow. The architecture is practical for an academic implementation and can be extended as conference size and requirements increase.

16. References

1. EasyChair. (2026). Conference Management. https://easychair.org/conference_management

2. EasyChair. (2026). General Information and Services. https://easychair.org/docs/general

3. OpenReview. (2025). Default Registration Form: profile and expertise confirmation. https://docs.openreview.net/reference/default-forms/default-registration-form

4. Association for Computing Machinery, SIGPLAN. (2026). Conference Submission Review Policy. https://sigplan-pages.sigplan.hosting.acm.org/Resources/Policies/Review/

5. Association for Computing Machinery, SIGSOFT. (2026). Policy on Program Committees. https://www2.sigsoft.org/policies/pcpolicy/

6. Committee on Publication Ethics (COPE). (2017; current guidance page). Ethical Guidelines for Peer Reviewers. https://doi.org/10.24318/2019.1.4

7. PLOS. (2026). Competing Interests for Reviewers: A Guide to Ethical Peer Review. https://explore.plos.org/reviewer-resources/competing-interests

8. Association for Computing Machinery. (2025). UAI 2025 Instructions for Reviewers and Area Chairs. https://www.auai.org/uai2025/instructions_for_reviewers_and_area_chairs

9. International Association for Dental, Oral, and Craniofacial Research. (2025). IADR Code of Ethics: Ethical Considerations for Peer Review. https://www.iadr.org/about/code-ethics
10. SAGE Publications. (2026). Peer Review Ethics. https://www.sagepub.com/journals/information-for-reviewers/peer-review-ethics